Introducing IaCConf 2025, the free virtual event for elevating your IaC skills Register Now

KUBERNETES

# How to Deploy Kubernetes Resources with Terraform

**Faisal Hashem**
27 Aug 2024 · 21 min read

Reviewed by: Flavius Dinu

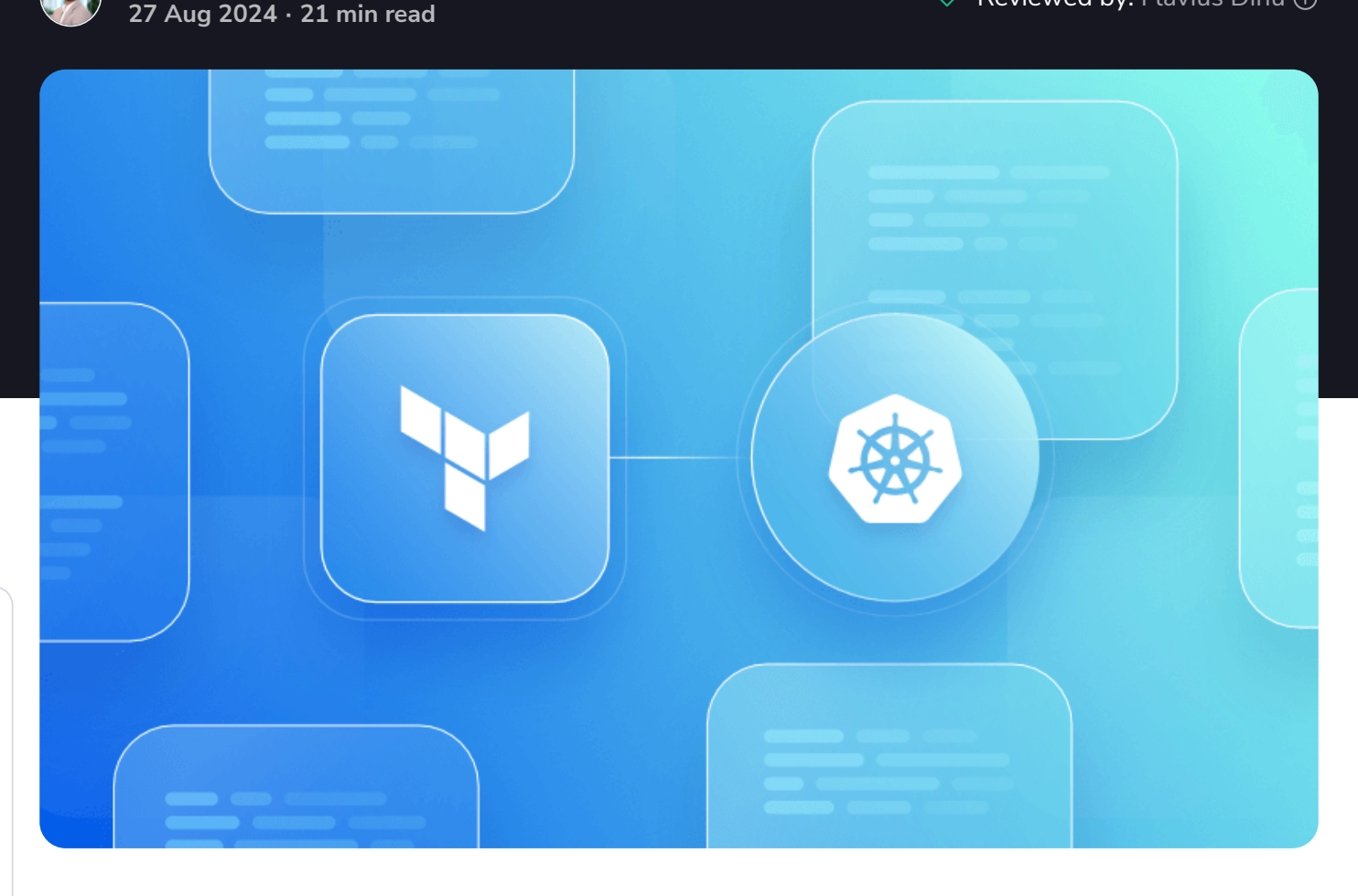

TABLE OF CONTENTS
Benefits of using Terraform with Kubernetes
1. Set up a Kubernetes cluster
2. Deploy a sample application to Kubernetes with Terraform
3. Automate Terraform and Kubernetes workflow in the CI/CD pipeline
Best practices for a successful Terraform Kubernetes deployment
Key points

🚀 Level Up Your Infrastructure Skills
You focus on building. We'll keep you updated. Get curated infrastructure insights that help you make smarter decisions.
Your email address
Subscribe

Kubernetes (K8s) is an open-source platform that automates the deployment and management of containerized applications. It streamlines operational tasks like scaling and managing containers, ensuring consistency across environments. By simplifying the process of launching and shutting down containers, Kubernetes addresses the complexities of manual container management.

Kubernetes also offers features like self-healing, service discovery, load balancing, and storage orchestration, all managed through YAML for easier updates.

Terraform, a Hashicorp infrastructure-as-code (IaC tool) by HashiCorp, complements Kubernetes by enabling automated, code-driven infrastructure management across various providers like Azure, AWS, and Google Cloud. Terraform's ability to plan deployments in advance ensures accuracy and reduces errors, making it a preferred choice for managing IaC.

What we will cover:

1. Benefits of using Terraform with Kubernetes
2. Setting up a Kubernetes cluster
3. Deploying a sample application to Kubernetes with Terraform
4. Automating Terraform and Kubernetes workflow in the CI/CD pipeline
5. Best practices for a successful Terraform Kubernetes deployment

## Benefits of using Terraform with Kubernetes

Using Terraform with Kubernetes allows for the infrastructure-as-code management of Kubernetes clusters and associated resources, making deployment consistent and repeatable. Terraform can provision Kubernetes clusters across different cloud providers, manage Kubernetes resources like services and deployments, and integrate with Kubernetes through a Kubernetes provider or by managing infrastructure like networking and storage.

Here are some of the key benefits of using Terraform to deploy your applications in a Kubernetes cluster:

1. **Consistency** — By using Terraform to deploy and manage both infrastructure and Kubernetes, you ensure consistency across your environment, simplifying version control and CI/CD processes for your applications and deployments.
2. **Scalability** — Terraform makes it easy to scale your Kubernetes cluster nodes and application components, allowing you to adjust resources as needed effortlessly.
3. **Automation** — With Terraform, you can build and manage your infrastructure and Kubernetes resources seamlessly. This allows you to spin up environments like DEV, QA, UAT, and PROD with just a click, fully automating the deployment of both infrastructure and applications.
4. **Cross-platform flexibility** — Terraform's provider-based architecture allows you to deploy and manage Kubernetes clusters across multiple cloud providers (AWS, Azure, GCP) or even in on-premises environments. Regardless of the cloud provider or environment, Terraform provides a consistent workflow for managing your Kubernetes infrastructure.
5. **Cost efficiency** — Terraform allows for fine-tuning and right-sizing of infrastructure according to actual application needs, leading to more efficient cost management. You can define and manage the exact resources needed for your Kubernetes cluster, such as nodes, storage, and network components. By specifying these resources upfront, you can avoid over-provisioning, which directly reduces operational costs.

## 1. Set up a Kubernetes cluster

To demonstrate how to deploy an application to a Kubernetes cluster using Terraform, we will use minikube. Then we will briefly discuss how to set up a Kubernetes cluster at an enterprise level using cloud provider plug-ins services like Azure AKS and Amazon EKS.

### minikube cluster setup

minikube is a free, open-source tool for setting up a local Kubernetes cluster on your machine. Although it's primarily useful for demo and development purposes, it's also a great way for new Kubernetes users to get hands-on experience and for developers to test deployments.

Before we start, your machine needs the following:

- 2 CPUs or more
- 2GB of free memory
- 20GB of free disk space
- Container or VM manager such as Docker, VirtualBox, VMware Fusion/Workstation, or Parallels

Now, let's start the installation process, which is very straightforward because you only have to run a single command to initiate the installation.

MacOS:

```
#Install cluster and kubectl
brew install minikube
brew install kubectl

#Start cluster
minikube start

#Validate cluster
minikube status
kubectl cluster-info
```

Windows:

```
#Install Chocolatey via Powershell
Set-ExecutionPolicy Bypass -Scope Process -Force; [System.Net.ServicePointManager]::SecurityProt

#Install minikube
choco install minikube

#Start cluster
minikube start

#Validate cluster
minikube status
kubectl cluster-info
```

Configure Alias (for simplicity):

```
#Initially in order to get pods you would run:
minikube kubectl -- get pods -A

#Create an alias in order to simplify and run 'kubectl':
alias kubectl="minikube kubectl --"

#Now you can run:
kubectl get pods -A
```

To stop the minikube instance:

```
#This will stop your Kubernetes cluster without affecting the deployed apps:
minikube pause

#Fully stop the Kubernetes cluster
minikube stop

#Delete the Kubernetes cluster
minikube delete --all
```

### Azure AKS cluster setup

We will now use Azure CLI to install the AKS cluster.

**Pre-requisites**

- Azure Subscription

```
#Install Azure CLI and Kubectl for Windows
choco install azure-cli
choco install kubernetes-cli

#Install Azure CLI and Kubectl for MacOS
brew install azure-cli
brew install kubernetes-cli

#Login using your Azure creds
az login

az account set --subscription "enter-your-subscription-id"

#Create resource group for your Kubernetes cluster
az group create --name my-kube-rg --location eastus2

#Create AKS cluster
az aks create --resource-group my-kube-rg --name theakscluster --node-count 1 --enable-addons mo

#Add Kube config with AKS cluster
az aks get-credentials --resource-group my-kube-rg --name theakscluster

#Validate AKS cluster is accessible and working
kubectl get nodes
```

### Amazon EKS Cluster Setup

We will use AWS CLI to deploy the EKS cluster.

**Pre-requisites**

- AWS account
- AWS user account with administrator role and access keys created

```
#Install kubectl, eksctl and AWS CLI for Windows
choco install kubernetes-cli
choco install eksctl
choco install awscli

#Install kubectl, eksctl and AWS CLI for MacOS
brew install kubernetes-cli
brew install eksctl
brew install awscli

#configure AWS CLI with your AWS Access Key ID, Secret Access Key and region
aws configure

#deploy EKS cluster
eksctl create cluster --name the-eks-cluster --region us-east-2 --nodegroup-name the-node-group

#Validate cluster
kubectl get nodes
```

## 2. Deploy a sample application to Kubernetes with Terraform

Now that our Kubernetes cluster is set up and accessible from our machine, we can begin configuring Terraform to deploy to a Kubernetes cluster, whether it's running locally with minikube, on Azure, or on AWS.

In simple terms, Terraform utilizes providers to point to the kubeconfig file path. This is how Terraform can communicate with your Kubernetes cluster and deploy the Kubernetes manifest resource regardless of its location.

### Local (minikube)

Let's start by setting up a single directory that includes a provider.tf file, which will contain the Terraform Kubernetes provider and the main.tf file with our main Terraform code for the Kubernetes resources.

Prior to this, make sure the kubeconfig file (~/.kube/config) contains your minikube cluster information and certificates. You can also validate by running `kubectl get all -A` and making sure you can retrieve the default pods.

In the following example, we will show how to deploy a sample nginx proxy server using Terraform.

*provider.tf*

```
terraform {
  required_providers {
    kubernetes = {
      source = "hashicorp/kubernetes"
      version = "2.30.0"
    }
  }
}

provider "kubernetes" {
  config_path = "~/.kube/config"
}
```

*main.tf*

```
resource "kubernetes_deployment" "test-deploy" {
  metadata {
    name = "terraform"
    labels = {
      test = "MyApp"
    }
  }

  spec {
    replicas = 3

    selector {
      match_labels = {
        test = "MyApp"
      }
    }

    template {
      metadata {
        labels = {
          test = "MyApp"
        }
      }

      spec {
        container {
          image = "nginx:1.21.6"
          name  = "nginx-terraform"

          resources {
            limits = {
              cpu    = "0.5"
              memory = "512Mi"
            }
            requests = {
              cpu    = "250m"
              memory = "50Mi"
            }
          }

          liveness_probe {
            http_get {
              path = "/"
              port = 80

              http_header {
                name  = "X-Custom-Header"
                value = "Awesome"
              }
            }

            initial_delay_seconds = 3
            period_seconds        = 3
          }
        }
      }
    }
  }
}
```

Now, from the directory where our provider.tf and main.tf configuration files are located, we'll test and verify that our application has been successfully deployed, by running our Terraform commands:

```
terraform init
terraform validate
terraform plan
terraform apply -auto-approve
```

You can also run the following to ensure the pods were deployed successfully:

```
MacBook-Pro:k8s-terraform fhashem$ kubectl get deploy
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
terraform   3/3     3            3           2m33s

MacBook-Pro:k8s-terraform fhashem$ kubectl get pod
NAME                         READY   STATUS    RESTARTS   AGE
terraform-988769666-7qssq    1/1     Running   0          3m20s
terraform-988769666-tnk82    1/1     Running   0          3m20s
terraform-988769666-vw9xs    1/1     Running   0          3m20s
```

Once you confirm the pods were deployed successfully, you can go ahead and destroy the Kubernetes resources.

```
terraform plan --destroy
terraform destroy
```

### Azure (AKS)

To successfully deploy Kubernetes resources in Azure, we first need an AKS cluster. In this setup, we'll use Terraform to deploy both the AKS cluster and the Kubernetes resources, showing how Terraform can serve as a unified tool for managing your infrastructure and application deployments.

To start, you'll need to create a service principal in your Azure tenant with the Contributor role assigned at the Subscription level (for simplicity). Then, retrieve the Tenant ID, Subscription ID, Client ID, and Client Secret, and set them as environment variables on your local machine. This will allow Terraform to communicate with your Azure tenant.

Run the following on your local machine with your Azure tenant's information:

```
export ARM_CLIENT_ID="00000000-0000-0000-0000-000000000000"
export ARM_CLIENT_SECRET="12345678-0000-0000-0000-000000000000"
export ARM_TENANT_ID="10000000-0000-0000-0000-000000000000"
export ARM_SUBSCRIPTION_ID="20000000-0000-0000-0000-000000000000"
```

Once that is complete, we will begin creating the Terraform files. To simplify things, the **main.tf** will include the Terraform code to deploy our AKS cluster and Kubernetes resource deployment.

**provider.tf**

We need to add both azurerm and Kubernetes providers since we will be deploying Azure resources and Kubernetes resources.

The host, client_certificate, client_key, and client_ca_certificate are needed here to grab our AKS cluster information and communicate via Terraform to the AKS cluster to deploy Kubernetes resources after the cluster is deployed:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "3.108.0"
    }

    kubernetes = {
      source = "hashicorp/kubernetes"
      version = "2.30.0"
    }
  }
}

provider "azurerm" {
  features {}
}

provider "kubernetes" {
  host = azurerm_kubernetes_cluster.aks.kube_config.0.host
  client_certificate = base64decode(azurerm_kubernetes_cluster.aks.kube_config.0.client_certific
  client_key = base64decode(azurerm_kubernetes_cluster.aks.kube_config.0.client_key)
  cluster_ca_certificate = base64decode(azurerm_kubernetes_cluster.aks.kube_config.0.cluster_ca_
}
```

**main.tf**

This will deploy a resource group for our AKS cluster, AKS cluster, Kubernetes namespace, Kubernetes deployment, and Kubernetes service, and give you the load balancer IP to test accessing the cluster.

```
resource "azurerm_resource_group" "aks" {
  name     = "myAKSrg"
  location = "Central US"
}

#AKS Cluster
resource "azurerm_kubernetes_cluster" "aks" {
  name                = "fh-test-cluster"
  location            = azurerm_resource_group.aks.location
  resource_group_name = azurerm_resource_group.aks.name
  dns_prefix          = "myaksdns"

  default_node_pool {
    name       = "default"
    node_count = 1
    vm_size    = "Standard_DS2_v2"
  }

  identity {
    type = "SystemAssigned"
  }
}

#Kubernetes namespace to hold application
resource "kubernetes_namespace" "terraform-k8s" {
  metadata {
    name = "terraform-k8s"
  }
}

#Kubernetes deployment
resource "kubernetes_deployment" "nginx" {
  metadata {
    name      = "nginx"
    namespace = kubernetes_namespace.terraform-k8s.metadata[0].name
  }

  spec {
    replicas = 1

    selector {
      match_labels = {
        app = "nginx"
      }
    }

    template {
      metadata {
        labels = {
          app = "nginx"
        }
      }

      spec {
        container {
          name  = "nginx"
          image = "nginx:1.21.6"

          port {
            container_port = 80
          }
        }
      }
    }
  }
}

#Kubernetes service to access nginx webpage
resource "kubernetes_service" "nginx" {
  metadata {
    name      = "nginx"
    namespace = kubernetes_namespace.terraform-k8s.metadata[0].name
  }

  spec {
    selector = {
      app = kubernetes_deployment.nginx.spec[0].template[0].metadata[0].labels.app
    }

    port {
      port        = 80
      target_port = 80
    }

    type = "LoadBalancer"
  }
}

#Nginx Load Balancer to output Public IP to access nginx from Web Browser
output "nginx_load_balancer_ip" {
  value = kubernetes_service.nginx.status[0].load_balancer[0].ingress[0].ip
}
```

Run the following to deploy the Terraform code:

```
terraform init
terraform validate
terraform plan
terraform apply -auto-approve
```

Once the deployment is complete, let's check to make sure the Kubernetes resources have been successfully deployed. To access the cluster, we will need to first add the AKS cluster config to our local machines *~/.kube/config*.

Run the following commands to add the kubeconfig:

```
#Need to login to your Azure account

az login

#This adds your AKS cluster config to your local machines ~/.kube/config so you can connect to t

az aks get-credentials --resource-group myAKSrg --name fh-test-cluster

#This is applicable if you followed the minikube demo and have set a minikube kubectl alias, nee

unalias kubectl

#Run Kubectl commands to ensure resources have been deployed

kubectl get deployment -n terraform-k8s
kubectl get pods -n terraform-k8s
```

You should get the following output:

```
kubectl get deployment -n terraform-k8s
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           9m22s

kubectl get service -n terraform-k8s
NAME    TYPE           CLUSTER-IP     EXTERNAL-IP     PORT(S)        AGE
nginx   LoadBalancer   10.x.x.x      172.170.55.37   80:30302/TCP   9m24s
```

You can also access the public IP address to ensure all resources were successfully deployed. You should see the following:

We can go ahead and destroy the AKS and kubernetes resources now:

```
terraform plan --destroy
terraform destroy
```

## Amazon AWS EKS

Now, let's walk through setting up our Terraform configuration for an AWS EKS cluster, along with all the necessary network and policy configurations. This process will resemble what we did for minikube and Azure AKS. However, the Terraform code for the EKS cluster will require more AWS resources than an Azure AKS cluster. We will cover this in detail below.

To begin, let's create an IAM user with administrator access (for simplicity) and generate the access keys. Then we'll set up our environment variables with your access key and secret key.

```
export AWS_ACCESS_KEY_ID="accesskey"
export AWS_SECRET_ACCESS_KEY="secretkey"
export AWS_REGION="us-east-2"
```

Once we create the environment variables, we can start working on the provider.tf and main.tf configuration files. To avoid conflicts, make sure this is not created on the same directory as the minikube/azure aks.

*provider.tf*

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "5.54.1"
    }
    kubernetes = {
      source = "hashicorp/kubernetes"
      version = "2.30.0"
    }
  }
}
provider "aws" {
}
```

*main.tf*

```
#Network (All of the following are related to AWS EKS Networking)
resource "aws_vpc" "eks_vpc" {
  cidr_block = "172.20.0.0/16"
  tags = {
    Name = "eks-vpc"
  }
}

resource "aws_subnet" "public_eks_subnet" {
  count         = 2
  vpc_id            = aws_vpc.eks_vpc.id
  cidr_block        = element(["172.20.1.0/24", "172.20.2.0/24"], count.index)
  availability_zone = element(["us-east-2a", "us-east-2b"], count.index)
  map_public_ip_on_launch = true
  tags = {
    Name = "eks-public-subnet-${count.index}"
  }
}

resource "aws_subnet" "private_eks_subnet" {
  count             = 2
  vpc_id            = aws_vpc.eks_vpc.id
  cidr_block        = element(["172.20.3.0/24", "172.20.4.0/24"], count.index)
  availability_zone = element(["us-east-2a", "us-east-2b"], count.index)
  map_public_ip_on_launch = true
  tags = {
    Name = "eks-private-subnet-${count.index}"
  }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.eks_vpc.id
  tags = {
    Name = "my-eks-cluster"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.eks_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = {
    Name = "my-eks-cluster-public"
  }
}

resource "aws_route_table_association" "public" {
  count          = 2
  subnet_id      = element(aws_subnet.public_eks_subnet[*].id, count.index)
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "eks_cluster_sg" {
  name        = "my-eks-cluster-eks-cluster-sg"
  description = "EKS cluster security group"
  vpc_id      = aws_vpc.eks_vpc.id
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "my-eks-cluster-eks-cluster-sg"
  }
}

resource "aws_security_group" "eks_node_sg" {
  name        = "my-eks-cluster-eks-node-sg"
  description = "EKS worker node security group"
  vpc_id      = aws_vpc.eks_vpc.id
  ingress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    security_groups = [
      aws_security_group.eks_cluster_sg.id
    ]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "my-eks-cluster-eks-node-sg"
  }
}

#IAM Roles/Policies - All of the following are related to IAM Role and Policies
resource "aws_iam_role" "eks_role" {
  name = "eks-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "eks.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "eks_policy" {
  role       = aws_iam_role.eks_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

resource "aws_iam_role" "eks_node_group_role" {
  name = "eks-node-group-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "eks_node_group_policy" {
  role       = aws_iam_role.eks_node_group_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {
  role       = aws_iam_role.eks_node_group_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

resource "aws_iam_role_policy_attachment" "eks_ecr_policy" {
  role       = aws_iam_role.eks_node_group_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

#EKS Cluster and Node Group deployment
resource "aws_eks_cluster" "eks_cluster" {
  name     = "my-eks-cluster"
  role_arn = aws_iam_role.eks_role.arn
  vpc_config {
    subnet_ids = aws_subnet.private_eks_subnet[*].id
    security_group_ids = [aws_security_group.eks_cluster_sg.id]
  }
  depends_on = [aws_iam_role_policy_attachment.eks_policy]
}

resource "aws_eks_node_group" "eks_node_group" {
  cluster_name    = aws_eks_cluster.eks_cluster.name
  node_group_name = "my-node-group"
  node_role_arn   = aws_iam_role.eks_node_group_role.arn
  subnet_ids      = aws_subnet.private_eks_subnet[*].id
  scaling_config {
    desired_size = 1
    max_size     = 2
    min_size     = 1
  }

  instance_types = ["t3.medium"]
  remote_access {
    ec2_ssh_key = "aws-key"  # Replace with your key pair name
  }

  update_config {
    max_unavailable = 1
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_node_group_policy,
    aws_iam_role_policy_attachment.eks_cni_policy,
    aws_iam_role_policy_attachment.eks_ecr_policy
  ]

}

data "aws_eks_cluster" "eks_cluster" {
  name = aws_eks_cluster.eks_cluster.name
}

data "aws_eks_cluster_auth" "eks_cluster" {
  name = aws_eks_cluster.eks_cluster.name
}

#Kubernetes provider for Terraform to connect with AWS EKS Cluster
provider "kubernetes" {
  host                   = data.aws_eks_cluster.eks_cluster.endpoint
  token                  = data.aws_eks_cluster_auth.eks_cluster.token
  cluster_ca_certificate = base64decode(data.aws_eks_cluster.eks_cluster.certificate_authority[
}

#Kubernetes resources in Terraform
resource "kubernetes_namespace" "terraform-k8s" {
  metadata {
    name = "terraform-k8s"
  }
}

resource "kubernetes_deployment" "nginx" {
  metadata {
    name      = "nginx"
    namespace = kubernetes_namespace.terraform-k8s.metadata[0].name
  }

  spec {
    replicas = 1

    selector {
      match_labels = {
        app = "nginx"
      }
    }

    template {
      metadata {
        labels = {
          app = "nginx"
        }
      }

      spec {
        container {
          name  = "nginx"
          image = "nginx:1.21.6"

          ports {
            container_port = 80
          }
        }
      }
    }
  }
}

resource "kubernetes_service" "nginx" {
  metadata {
    name      = "nginx"
    namespace = kubernetes_namespace.terraform-k8s.metadata[0].name
  }

  spec {
    selector = {
      app = kubernetes_deployment.nginx.spec[0].template[0].metadata[0].labels.app
    }

    port {
      port        = 80
      target_port = 80
    }

    type = "LoadBalancer"
  }
}

#Output Load Balancer IP to access from browser
output "nginx_load_balancer_ip" {
  value = kubernetes_service.nginx.status[0].load_balancer[0].ingress[0].ip
}
```

Run the following to deploy the Terraform code:

```
terraform init
terraform validate
terraform plan
terraform apply -auto-approve
```

Once the deployment is complete, let's check to make sure the EKS cluster and Kubernetes resources have been successfully deployed. Go to the AWS portal and confirm that you see all the AWS resources.

To access the Kubernetes resources and validate they have been deployed successfully, we will need to add the EKS cluster config to our local machine *~/.kube/config*. To do this, you will need AWS CLI and Kubectl installed.

Run the following commands:

```
#Need to make sure AWS account is configured on your machine

aws configure

#This adds your EKS cluster config to your local machines ~/.kube/config so you can connect to

aws eks --region us-east-2 update-kubeconfig --name my-eks-cluster

#Check to make sure you're connected to you AWS EKS cluster
kubectl config current-context

#Run Kubectl commands to ensure resources have been deployed

kubectl get deployment -n terraform-k8s
kubectl get pods -n terraform-k8s
```

The output should be similar to below:

```
kubectl get deployment -n terraform-k8s
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           9m22s

kubectl get service -n terraform-k8s
NAME    TYPE           CLUSTER-IP     EXTERNAL-IP     PORT(S)        AGE
nginx   LoadBalancer   10.x.x.x      172.170.55.37   80:30302/TCP   9m24s
```

You can also access the public IP address to ensure all resources were successfully deployed, should see the following:

We can go ahead and destroy the EKS and Kubernetes resources now:

```
terraform plan --destroy
terraform destroy —-auto-approve
```

💡 **You might also like:**

- Terraform vs. Kubernetes Comparison
- How to Automate Terraform Deployments and Infrastructure Provisioning
- How to Maintain Operations Around Kubernetes Cluster

## 3. Automate Terraform and Kubernetes workflow in the CI/CD pipeline

Using CI/CD with Terraform for Kubernetes deployment streamlines infrastructure management and application deployment.

Continuous integration (CI) ensures that every code change is tested and integrated into the main branch, catching issues early in the development cycle. Continuous deployment (CD) maintains consistency across your infrastructure by using Terraform as your standard IaC tool.

Automating with CI/CD reduces the risk of human error, speeds up the deployment process, and allows for smaller, more frequent updates that are easier to manage and troubleshoot.

### Using Jenkins pipeline to deploy an app in Kubernetes with Terraform

We will use Jenkins CI/CD to demonstrate how Terraform can be utilized to deploy Kubernetes resources in our pipeline. For this setup, we will host the Jenkins CI/CD on a Docker container, and the Kubernetes cluster will be on my minikube cluster on our local machine.

If you prefer to use this cluster setup, please refer to the minikube cluster setup section for instructions on setting up a minikube cluster on your local machine. This should also apply to any other Jenkins instance and Kubernetes cluster.

On a high level, the Jenkins pipeline in our Docker container will check out the GIT repository for our Terraform code and run the code against our minikube cluster to deploy our Kubernetes resources.

**Prerequisites (on a local machine):**

- Terraform installed
- minikube installed
- Kubectl installed and configured with minikube by creating a Docker image that contains all the packages we need to ensure Jenkins can install and run our Terraform cluster
- Docker installed
- GitHub repository to host Terraform code

**Create a Docker image**

We will start by creating a Docker image that contains all the packages we need to ensure Jenkins can install and run our Terraform job correctly. Once we have the image, we will create a container to host our Jenkins instance.

*Dockerfile*

```
#default Jenkins image
FROM jenkins/jenkins:lts

ENV DEBIAN_FRONTEND=noninteractive

USER root

#Need these packages installed to install Kubectl and Terraform
RUN apt-get update && \
    apt-get install -y \
    wget \
    unzip \
    curl \
    git \
    gnupg \
    ca-certificates && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

#Install terraform
RUN wget https://releases.hashicorp.com/terraform/1.5.3/terraform_1.5.3_linux_amd64.zip && \
    unzip terraform_1.5.3_linux_amd64.zip && \
    mv terraform /usr/local/bin/ && \
    rm terraform_1.5.3_linux_amd64.zip

#Install kubectl
RUN curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/
    install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl && \
    rm kubectl

USER jenkins

#Access jenkins on port 8080
EXPOSE 8080

CMD ["jenkins.sh"]
```

Now from the directory you created the Dockerfile in, run the following:

```
#To create the docker image
docker build . -t jenkins-image:v1

#Validate image exists
docker images
```

We will run the Jenkins instance using this image. To ensure Jenkins can communicate with our minikube instance, we will mount our ~/.kube/config and /Users/your username/.minikube directory to the Jenkins container.

**Note**: The directory path for the minikube certificates will be different if you are using Windows, this will work for MacOS.

```
#Create Jenkins container
docker run -d -p 8080:8080 -p 50000:50000 -v ~/.kube:/var/jenkins_home/.kube -v jenkins_home:/va

#Validate terraform and kubectl are working on container
docker exec jenkins-instance kubectl —-help
docker exec jenkins-instance terraform —-help

#Grab the password, we will need this to login to Jenkins
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

**Set up the Jenkins instance**

Follow these steps to set up your Jenkins instance:

1. Open your web browser and go to http://localhost:8080 to access your Jenkins CI/CD.
2. Once you are prompted to enter your admin password, enter the password we grabbed from the previous step.
3. Go through the setup pages to install the recommended plugins and create your admin user, which you will use to log in from then on.
4. Go to **Manage Jenkins** > **Plugins** > **Available plugins** and install the following plugins:
   - Kubernetes
   - Kubernetes CLI
   - Terraform

We should be all set to start creating our Terraform resources and our Jenkins pipeline.

Let's start setting up our Terraform code, which will contain the Kubernetes resources we will deploy to our minikube cluster through Jenkins. You will need to host your Terraform code on a GitHub repository. We will not cover that in this article, but you just need to make sure the repository is public so we can access it from our Jenkins pipeline.

In your GitHub repository, create the following main.tf file:

```
provider "kubernetes" {
  config_path = "~/.kube/config"
}

#Kubernetes namespace to hold application
resource "kubernetes_namespace" "terraform-k8s" {
  metadata {
    name = "terraform-k8s"
  }
}
```

```
resource "kubernetes_deployment" "test-deploy" {
  metadata {
    name = "terraform"
    namespace = "terraform-k8s"
    labels = {
      test = "MyApp"
    }
  }

  spec {
    replicas = 3

    selector {
      match_labels = {
        test = "MyApp"
      }
    }

    template {
      metadata {
        labels = {
          test = "MyApp"
        }
      }

      spec {
        container {
          image = "nginx:1.21.6"
          name  = "nginx-terraform"

          resources {
            limits = {
              cpu    = "0.5"
              memory = "512Mi"
            }
            requests = {
              cpu    = "250m"
              memory = "50Mi"
            }
          }

          liveness_probe {
            http_get {
              path = "/"
              port = 80

              http_header {
                name  = "X-Custom-Header"
                value = "Awesome"
              }
            }

            initial_delay_seconds = 3
            period_seconds        = 3
          }
        }
      }
    }
  }
}
```

**Create the Jenkins pipeline**

We will now return to our Jenkins instance and create the pipeline. For this example, we will add the pipeline solely to deploy the Kubernetes resources using Terraform. However, this can be used alongside the other CI/CD steps in your pipeline.

Go to your Jenkins instance and create a new item, select **Pipeline,** and name it "k8s-terraform-pipeline".

Add the following pipeline script:

```
pipeline{
     agent any

     stages{
        stage('Git Checkout'){
            steps{
                git branch: 'main', url: 'https://github.com/yourusername/repo_name'
            }
        }

        stage('Terraform init'){
            steps{
                dir("terraform/spacelift/terraform-k8s"){
                     sh 'terraform init'
                }
            }
        }

        stage('Terraform plan'){
            steps{
                dir("terraform/spacelift/terraform-k8s"){
                     sh 'terraform plan'
                }
            }
        }

        stage('Terraform apply'){
            steps{
                dir("terraform/spacelift/terraform-k8s"){
                     sh 'terraform apply --auto-approve'
                }
            }
        }
    }
}
```

Now save the pipeline and run the build.

Once the pipeline runs successfully, go to your terminal and run the following to ensure the application was deployed successfully to your Kubernetes cluster through the Jenkins CI/CD pipeline:

```
kubectl get deployment -n terraform-k8s
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           9m22s

kubectl get service -n terraform-k8s
NAME    TYPE           CLUSTER-IP     EXTERNAL-IP     PORT(S)        AGE
nginx   LoadBalancer   10.x.x.x       172.170.55.37   80:30302/TCP   9m24s
```

## Using Spacelift stack dependencies to deploy an app in Kubernetes with Terraform

To simplify your CI/CD processes, you can also leverage Spacelift stack dependencies.

The stack dependencies feature allows you to define and manage the relationships between different stacks, ensuring that your pipeline steps run in the correct order and all dependencies are resolved before proceeding to specific steps. This is particularly useful in complex infrastructure environments where multiple components — such as account creation and assignments, networking, databases, compute resources, Kubernetes clusters, monitoring, and alerting — must be provisioned in a precise sequence.

You can easily pass information between stacks, which not only minimizes the risk of errors but also streamlines the deployment process, making it more efficient and reliable. When using stack dependencies in CI/CD pipelines, you achieve a smoother and more controlled deployment.

In this example, we will demonstrate using stack dependencies to deploy Kubernetes resources to an Azure AKS cluster.

We will break this process into two stacks: The first stack will focus on deploying an AKS cluster with Terraform, and the second stack will handle deploying Kubernetes resources into the AKS cluster. We will establish a dependency from stack 1 to stack 2, passing the kube config generated by the AKS cluster (stack 1) as an output to the Kubernetes stack (stack 2).

**Prerequisites**

- Spacelift account (you can sign up for a 14-day free trial if needed)
- Azure subscription
- GitHub repo

**Configure cloud integration**

In your Spacelift account, go to the **Cloud Integrations** side tab, click on **Azure** and **Create Integration**.

Add your Azure Tenant ID and Subscription ID here, and then click **Create Integration**.

You will get a prompt to **Provide Consent**. Click that and wait a few minutes.

Go to your Azure portal, click on **Microsoft Entra ID** > **Enterprise Applications** and search for 'spacelift'. Confirm you can see the application there. If you do not see the application, validate that you have administrator permissions in this Azure subscription and try again.

Now, in your Azure Portal, go to your **Subscription** > **IAM** > **Add Role Assignment**. Select the **Privileged administrator roles** tab and select **Contributor**. Assign this role to the enterprise application Spacelift. Review the changes and click on **Review and assign**.

**Set up Git repositories for Terraform and Kubernetes resources**

We will create two separate directories in the GitHub repository. One will have the Terraform code to deploy our Azure AKS cluster deployment (we will use the terraform k8s module). The second directory will include the code for our Kubernetes manifest.

From the repository's root directory, create a directory named 'tf'. Add the following main.tf file:

```
provider "azurerm" {
 features {}
}

resource "azurerm_resource_group" "azrg" {
 name     = "az-rg"
 location = "centralus"
}

module "aks" {
 source = "github.com/flavius-dinu/terraform-az-aks.git?ref=v1.0.12"
 kube_params = {
   kube1 = {
     name                = "kube1"
     rg_name             = azurerm_resource_group.azrg.name
     rg_location         = "centralus"
     dns_prefix          = "kube"
     identity            = [{}]
     enable_auto_scaling = false
     node_count          = 1
     np_name             = "kube1"
     export_kube_config  = false
     tags                = {}
   }
 }
}

output "kube_config" {
 value     = module.aks.kube_config["kube1"]
 sensitive = true
}
```

Again, in the repository's root directory, create another directory name it 'k8s'.Then, add the following nginx.yaml file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
spec:
 replicas: 2
 selector:
   matchLabels:
     app: nginx
 template:
   metadata:
     labels:
       app: nginx
   spec:
     containers:
     - name: nginx
       image: nginx:1.21.1
       ports:
       - containerPort: 80
```

Once you have created the files and set them up in your Git repository, you can start working on creating the stacks that will reference these directories.

**Set up Terraform (Azure AKS Cluster) and Kubernetes stacks with dependencies**

**Terraform stack creation**

On Spacelift's home page, click **Create Stack.**

Name your stack "**azure-aks-terraform**".

If you already logged in using your Github account to Spacelift, you should see your Github repositories. Select your **Repository**, **Branch**, and **Project Root,** which should point to the '**tf**' directory we created earlier in your Git repository.

If you did not log in to Spacelift using your GitHub account, you can log in here, point to your Git repository, and add your project root.

On the **Choose Vendor** screen, make sure **Terraform / OpenTofu** is selected, and everything else should be set to default.

Keep the rest of the screens as default, and click **Confirm**.

Now, we will trigger this stack to create the outputs for our AKS cluster and insert these outputs into the Kubernetes stack dependencies. Go to the azure-aks-terraform stack and click **Trigger**.

This should start creating all your Azure resources via Terraform. After the Terraform Plan is complete, you will be prompted to **Confirm** the run (Terraform Approve). Make sure to click **Confirm**.

Once the stack is complete, validate that the Azure resources and outputs are created successfully.

You can also check the Azure Portal and make sure the resources were created successfully.

**Kubernetes stack creation**

Let's return to Spacelift's homepage and create the second stack for Kubernetes resource deployment. You can name this one "Kubernetes."

For the Git source code, we will point this to the same repository and branch as the previous stack, but the project root will point to "k8s"'instead.

On the **Choose vendor** screen, select **Kubernetes** and leave everything else as default.

Let's skip to the **Add hooks** screen. We will add a "Before Initialization" workflow step to process the outputs from the first stack. We will explain more later, but for now add the following commands to the **Before** window:

```
mkdir /mnt/workspace/.kube
printf "%s\n" "$kubeconfig" > /mnt/workspace/.kube/config
```

Let's skip to the summary and click **Confirm**.

Go to your 'azure-aks-terraform' stack, click **Settings**, and click **Integrations** there. Select your Azure subscription here (you should be able to click the drop-down and have it auto-populate the Subscription ID) and make sure to assign it read/write permissions.

**Output/dependencies**

In the 'azure-aks-terraform' stack, go to the **Dependencies** tab and under **Depended on by,** click **Add Dependencies**. Select **Kubernetes** and click **Add**.

On the same screen, click **Add output reference**. Select 'kube_config' and enter 'kubeconfig' as the

**Input name**. This will insert the Terraform Azure AKS cluster output we placed in our Terraform code into the Kubernetes stack.

You can now go to the Spacelift homepage. Under stacks select 'azure-aks-terraform' stack and click **Trigger**.

This will run the Terraform stack first (it should not add anything new because we ran this earlier) and place the downstream stack for Kubernetes on "Queued" status. You will need to **Confirm** the Terraform deployment. Once the stack has finished running, the second stack for Kubernetes will get triggered and create the *nginx application* on the AKS cluster.

You can also validate the Kubernetes deployment was successful on the Azure Portal or through kubectl.

And that's it!  You have successfully deployed Kubernetes resources to an Azure AKS cluster via Terraform using Spacelift stack dependencies.

If you want to learn more about Spacelift, create a free account today, or book a demo with one of our engineers.

## Best practices for a successful Terraform Kubernetes deployment

Let's review some best practices for using Terraform to deploy Kubernetes resources:

| | |
|---|---|
| **Modularize your Kubernetes Terraform code** | By modularizing your resources, such as Kubernetes deployments, services, config maps, roles, and role bindings, you can simplify deployment and enable reusability. |
| **Utilize variables** | Always try to use variables in your Kubernetes instead of hard coding any values. You can also pass in environmental, regional, and other values from your pipeline down to your Kubernetes modules. |
| **State management** | Use the remote backend (Azure Storage Account or AWS S3 Bucket) to store your remote state file. Also, make sure to use state locking to prevent multiple changes to the terraform state at the same time. |
| **Outputs** | Utilize outputs as much as possible during cluster deployments using Terraform to get kubeconfig and other values, such as cluster endpoint, and use them within your Kubernetes resource deployments. |
| **IAM role management** | Use AWS IAM Roles and Policies to assign permissions to worker nodes in AWS. Also, use Azure AD integration with AKS to manage access to nodes, which is much easier than managing users through Kubernetes RBAC. |
| **Version control** | Manage your Kubernetes Terraform code in a version-controlled system such as GIT. To manage changes, use branch controls (feature branches, pull requests, approvals) and tags. |
| **Documentation** | Ensure clear and proper instructions are listed on the readme page and throughout the Terraform code to ensure clarity for new users. |

## Key points

Integrating Terraform with Kubernetes simplifies the deployment and management of containerized applications by ensuring consistency, scalability, and automation across environments. Kubernetes' flexibility across various platforms like Azure AKS, AWS EKS, and on-premises clusters enhances its utility when paired with Terraform. Automating CI/CD workflows with Terraform, using tools like Jenkins or Spacelift, further boosts deployment efficiency, promoting agile and secure infrastructure management.

**Note:** New versions of Terraform are placed under the BUSL license, but everything created before version 1.5.x stays open-source. OpenTofu is an open-source version of Terraform that expands on Terraform's existing concepts and offerings. It is a viable alternative to HashiCorp's Terraform, being forked from Terraform version 1.5.6.

**Manage Terraform and Kubernetes from Spacelift**

Spacelift allows you to automate, audit, secure, and continuously deliver your infrastructure.  It helps overcome common state management issues and adds several must-have features for infrastructure management.

Learn more

## Written by

**Faisal Hashem**

Faisal is an experienced IT professional with around 7 years in infrastructure and 3 years in DevOps, specializing in Azure, Azure DevOps, Terraform, Ansible, Kubernetes. Certified as an Azure Solutions Architect, focusing on cloud infrastructure and automation, ensuring efficient and scalable solutions. Passionate about sharing knowledge, he enjoys writing technical articles in his free time.

blog

## Read also

DOCKER 13 min read
**What Is a Dockerfile And How To Build It – Best Practices**

AZURE 9 min read
**How to Provision Azure AKS Cluster Using Terraform**

TERRAFORM 17 min read
**How to Create AWS IAM Policy Using Terraform (Tutorial)**

spacelift

Get our newsletter Subscribe

**Product**
- Documentation
- Why Spacelift
- How it works
- Spacelift Tutorial
- Pricing
- Customer Case Studies
- Integrations
- Security
- System Status
- Product Updates
- Test Pilot Program

**Company**
- About Us
- Careers
- Contact Sales
- Partners
- Media resources
- Accessibility

**Learn**
- Blog
- Atlantis Alternative
- Terraform Cloud Alternative
- Terraform Enterprise Alternative
- Spacelift for AWS
- Terraform Automation
- Achieve Terraform at Scale
- CI/CD for Infrastructure
- Drift Detection
- Newsletter

© 2025 Spacelift, Inc. All rights reserved